

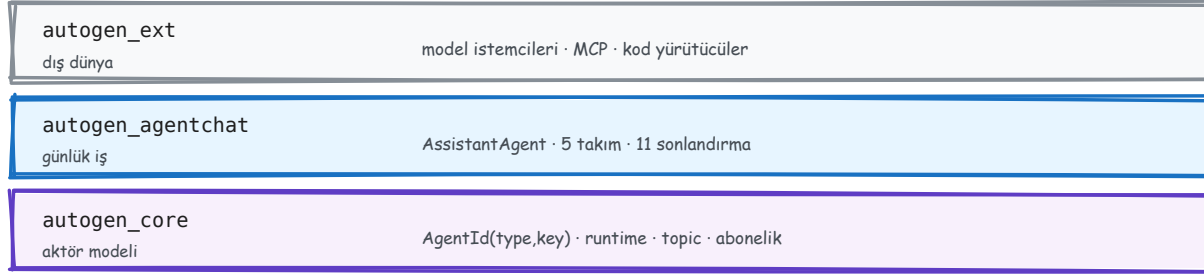
AutoGen, on altı şemada

Aktör modelinden tool döngüsüne: her slaytta bir mekanizma, nasıl çalıştığı ve neden önemli olduğu.

autogen-core · agentchat · ext v0.7.5

uzun anlatım: [docs/pdfs/slaytlar.pdf](https://docs/autogen/pdfs/slaytlar.pdf)

Üç katman



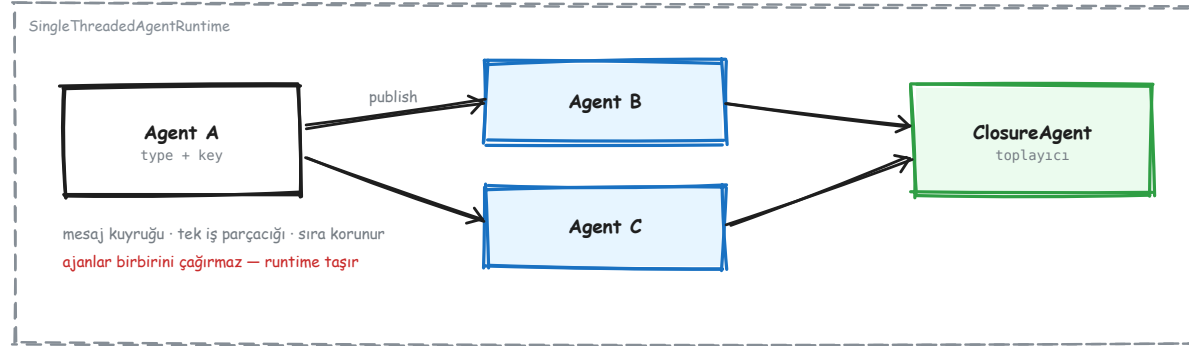
Ayıran şey en alt katman: ajanlar gerçekten aktör — kendi mailbox'ı olan, mesajı tipe göre yönlendiren birimler.

Yukarıdan başla; aşağı inmek her zaman mümkün.

autogen_core → autogen_agentchat → autogen_ext

AutoGen tek bir kütüphane değil, üst üste duran üç katman. En alttaki **autogen_core** **aktör modelini** kuruyor: ajanlar birbirini doğrudan çağırıyor, runtime'a mesaj veriyor. Ortadaki **autogen_agentchat** günlük işi kolaylaştırıyor: hazır ajan, beş takım tipi, on bir sonlandırma koşulu. Üstteki **autogen_ext** dış dünyaya bakıyor: model istemcileri, MCP, kod yürütücüler. Kural basit ve pratik: **yukarıdan başla**. AgentChat'in zaten çözdüğü bir problemi core'da yeniden çözmek, aynı işi daha az testle yapmak demektir.

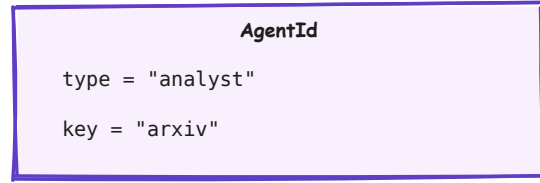
Aktör modeli



`runtime.publish_message(msg, topic)` — ajan ajanı çağırır

Bir ajan, başka bir ajanın nesnesini elinde tutmuyor ve metodunu çağırıyor. Runtime'a bir mesaj veriyor, teslimatı runtime yapıyor. Bunun iki bedeli var. Araya fazladan bir katman giriyor. Ve bir şey ters gittiğinde "kim kimi çağırdı" sorusunun cevabı yığın izinde görünmüyor, çünkü ortada bir çağrı zinciri yok. Karşılığında üç şey kazanıyorsun. Sisteme yeni bir ajan eklemek, çağırın kodu **hiç değiştirmiyor**. Bütün mesajlar tek bir noktadan geçtiği için müdahale ve ölçüm oraya bir kez takılıyor. Ve aynı sınıftan istediğin kadar örnek doğurmak bedava geliyor.

Kimlik: bir şey değil, iki şey



type = DAVRANIŞ (hangi sınıf)
key = ÖRNEK (hangi kopya)



aynı sınıf,
üç ayrı örnek,
üç ayrı durum

örnek talep üzerine doğar — önceden kaydedilmez

AgentId = (type, key) — davranış ve örnek

Her ajanın kimliği iki parçadan oluşuyor. **type** hangi sınıf olduğunu söylüyor; **key** ise o sınıfın hangi kopyası olduğunu. Fark önemli, çünkü runtime'a kaydettiğin şey yalnızca **type**. Key kaydedilmiyor; ilk kez o key'e mesaj gittiğinde örnek kendiliğinden doğuyor. Yani "üç ajan kaydettim" yanlış bir cümle. **Bir tip** kaydettin, üç örnek doğdu. Günlük hayatta bunun tek bir sonucu var ve onu bilmek yeter: **durumu key taşıyor**. Aynı key'e ikinci kez mesaj gönderirsen aynı belleğe gidiyor. Farklı bir key yazarsan sıfırdan, hiçbir şey hatırlamayan yeni bir ajan alıyorsun.

İki iletişim biçimi, iki asimetri

send_message — DOĞRUDAN



- cevabı DÖNDÜRÜR
- hata **ÇAĞIRANA** fırlar
- tek alıcı, bilinen adres

publish_message — YAYIN



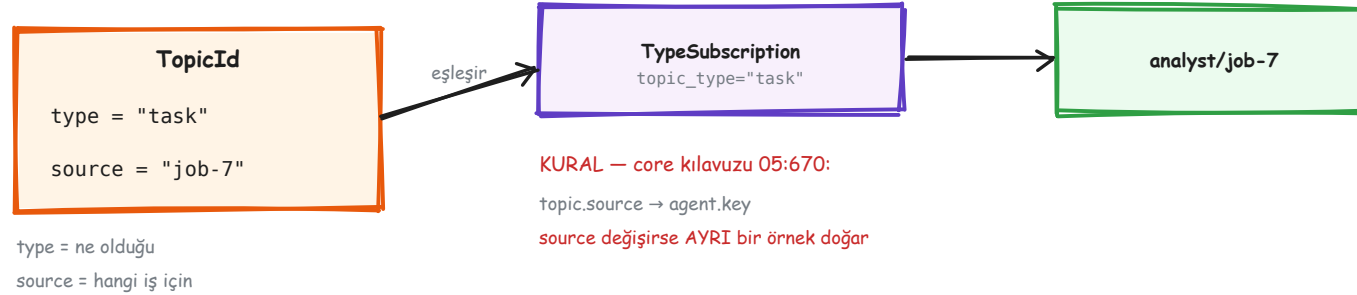
- None DÖNER — cevap yok
- hata yalnız **LOGLANIR**
- 0 abone de geçerli bir sonuç

Bu asimetri ölçüldü: yayında düşen bir handler sessizce düşer.

send → cevap döner, hata fırlar · publish → None döner, hata loglanır

`send_message` sıradan bir fonksiyon çağırısı gibi davranıyor: tek bir alıcı var, dönüş değeri var, ve içeride bir hata olursa hata sana kadar geliyor. `publish_message` ise bir duyuru. Kaç dinleyici olduğunu bilmiyorsun, dönüş değeri yok, ve **handler içinde patlayan istisna sana ulaşmıyor**; yalnızca loglanıyor. Üstelik hiç abone olmaması da hata sayılmıyor. Yayın başarılı kabul ediliyor. Yani "aboneliği yazmayı unuttum" hatası da sessiz kalıyor. Bu iki asimetri birleştğinde ortaya şu tablo çıkıyor: bir dal sessizce ölüyor, ve onu bekleyen toplayıcı sonsuza kadar bekliyor.

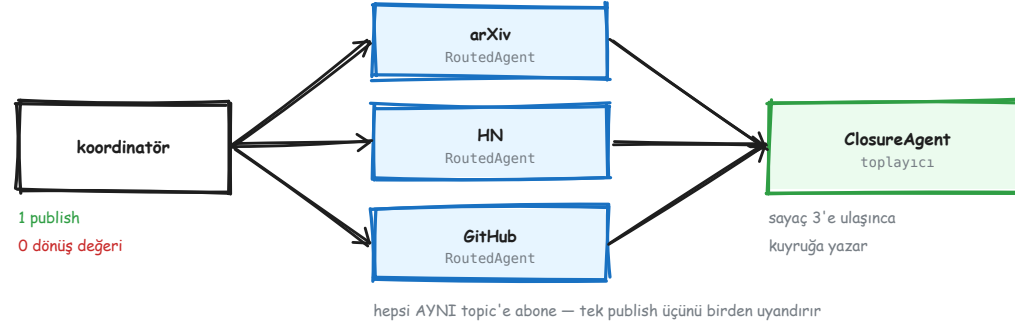
Topic: kanal adı değil, iki parçalı adres



topic.source → agent.key (core kılavuzu 05:670)

Topic da iki parçalı. **type** mesajın ne olduğunu söylüyor, **source** hangi iş için olduğunu. Abonelik *type*'a yapıyor. Ama mesajın hangi *örneğe* teslim edileceğini **source** belirliyor, hiçbir dönüşüm olmadan: `topic.source` doğrudan `agent.key` oluyor. Bunun sonucu, farkında olmazsan sinsi. Source'u her istekte değiştirirsen her istekte **yepyeni bir ajan örneği** doğuyor. Önceki örneğin belleği silinmiyor, sadece *ulaşamaz* hale geliyor. Ve bu daha kötü, çünkü hiçbir hata çıkmıyor: sistem çalışıyor, ajanlar cevap veriyor, ama hiçbiri bir öncekini hatırlamıyor.

Fan-out / fan-in



eşzamanlılık ajandan değil, ABONELİKTEN geliyor

Üç analist aynı topic tipine abone olduğu için tek bir yayın için birden uyandırıyor, ve runtime için paralel koşturuyor. Dikkat: ortada "paralel çalıştır" diyen bir çağrı yok. Paralellik, aynı duyuruyu birden fazla kişinin dinlemesinin doğal sonucu. Sonuçları toplamak içinse **ayrı bir ajan** gerekiyor, ve sebebi mekanik: `publish_message` hiçbir şey döndürmüyor. Dönüşü olmayan bir çağrıdan sonuç toplayamazsın. Kaç dal beklediğini de çerçeve saymıyor, **sen** sayıyorsun. Bu yüzden sayacı `finally` içinde artırmak zorunlu: dal çökse bile sayaç ilerlemezse toplayıcı hiç uyanmaz.

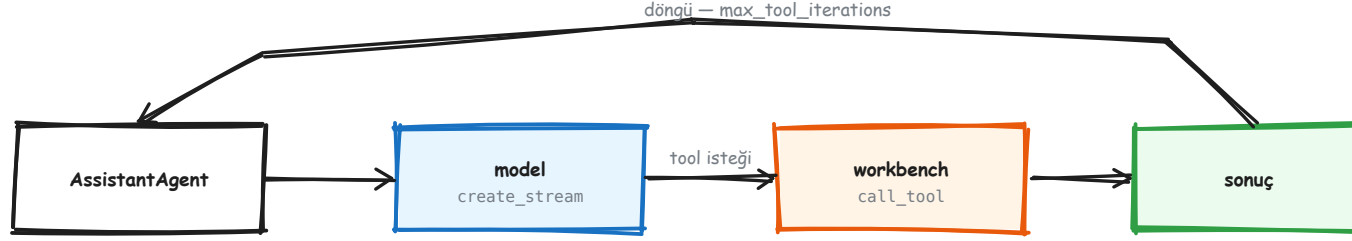
Müdahale – kapının tek atası



InterventionHandler → DropMessage

Runtime'a takılan bir kanca. Her `on_send` ve `on_publish` buradan geçiyor, ve `DropMessage` döndürürsen mesaj yok oluyor. Alıcı hiç haberdar olmuyor. Ama bu **bir onay mekanizması değil**, ve farkı bilmek önemli. İnsana soru sormuyor. Gerekçe döndürmüyor. Kararın kaydını tutmuyor. Ve reddi ajana bildirmiyor. Son madde en can alıcısı: ajan neden reddedildiğini bilmediği için başka bir yol da deneyemiyor. Sadece cevabı gelmemiş bir çağrı görüyor. Gerçek bir onay kapısı bir katman yukarıda, **workbench düzeyinde** kurulmak zorunda. Bizim yaptığımız da bu.

AssistantAgent ve tool döngüsü



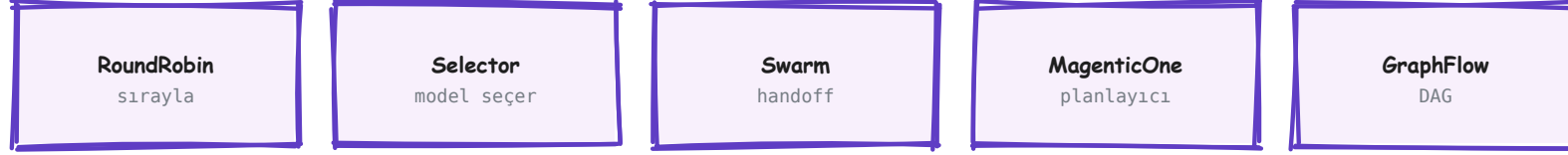
VARSAYILAN 1: model tool sonucunu **GÖRMEDEN** cevap verir

ölçüldü — bizde 6'ya çekildi

`max_tool_iterations` — varsayılan 1

Model bir tool çağırıyor, tool koşuyor, sonuç dönüyor. Ve varsayılan ayarda tur **tam orada bitiyor**. Yani model, tool'un bulduğu sonucu **hiç görmüyor**. Kullanıcıya giden cevap, tool çağırılmamış gibi yazılmış oluyor. Hiçbir hata çıkmıyor. Loga bakarsın: tool çağırılmış, sonuç dönmüş, her şey yolunda görünüyor. Ama cevap yanlış, çünkü eksik olan çağrı değil, **ikinci model turu**. Zincirleme davranış — "ara, bulduğunu oku, ona göre ikinci aramayı yap" — bu ayarla sessizce imkânsız. Bu destedeki en pahalı tek satır bilgi bu.

Beş takım – sırayı kim belirliyor



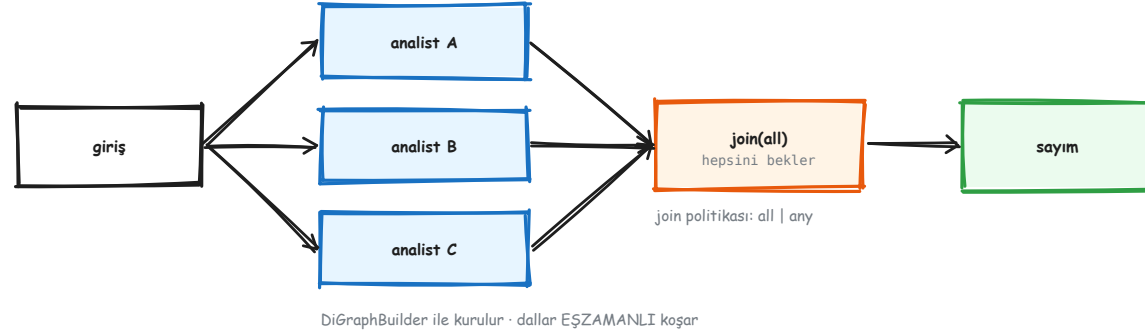
Beşi de aynı arayüz: `run()` / `run_stream()` → `TaskResult`

Taramamız *GraphFlow* kullanıyor — eşzamanlı dal + `join(all)`

aralarındaki tek fark: sıra kararını kim veriyor

Beş takım tipi var ve hepsi aynı işi yapıyor: birden çok ajanı sırayla konuşturmak. Aralarındaki tek fark, sıraya kimin karar verdiği. **RoundRobin** sabit bir döngü izliyor. **Selector** her turda modele soruyor. **Swarm** kararı ajanın kendisine bırakıyor. **MagenticOne** bir planlayıcıya veriyor. **GraphFlow** önceden çizilmiş bir grafiği takip ediyor. Maliyet farkı da buradan doğuyor. Selector her turda **iki** model çağrısı yapıyor: biri kimin konuşacağını seçmek için, biri asıl iş için. Sıra zaten belliyse bu ödenmemesi gereken bir maliyet. İyi haber: beşi de aynı arayüzü sunuyor, yani takım değiştirmek çağırılan kodu değiştirmiyor. Denemek ucuz.

GraphFlow – boru hattını çizmek



`DiGraphBuilder → add_edge(...) → join(all)`

Adımların sırası zaten belliyse, sırayı modele sordurmanın anlamı yok. GraphFlow'da grafiği çiziyorsun ve koşturuyorsun. Aynı düğüme birden fazla ok giriyorsa orası bir **birleşme noktası**. Varsayılan politika `all`, yani bütün dallar bitene kadar bekliyor. Kazandırdığı üç şey var. Eşzamanlılık bedava geliyor, çünkü aynı düğümden çıkan dallar paralel koşuyor. Fazladan model çağrısı yok, çünkü sırayı kimseye sormuyorsun. Ve **grafiğin kendisi bir belge**: sistemin ne yaptığı, kodun kendisine bakınca görünüyor.

Mesaj mı, olay mı

TextMessage	düz metin
StructuredMessage	pydantic şema
ToolCallRequestEvent	model tool istedi
ToolCallExecutionEvent	tool kaştı
ModelClientStreamingChunkEvent	token akışı
TaskResult	bitiş + stop_reason

Ayrım: mesaj konuşmanın parçası, olay ise ne olduğunun anlatımı.

mesaj = konuşmanın parçası · olay = ne olduğunun anlatımı

Ayrım kulağa akademik geliyor ama tamamen pratik. **Mesajlar** bağlama giriyor, yani bir sonraki model çağrısında tekrar gönderiliyor ve tekrar para ediyor. **Olaylar** yalnız gözlem için; bağlama hiç girmiyor. `StructuredMessage` bunun üstüne bir pydantic şeması taşıyor. Yani modelin cevabını serbest metin olarak ayrıştırmak yerine, doğrudan tipli bir nesne alıyorsun. Ve bir tuzak: akışın **son elemanı** bir olay değil, `TaskResult` 'ın kendisi. Döngüde tip kontrolü yapmazsan onu da bir olay sanıp işlersin, sonra da "sonuç hiç gelmedi" diye ararsın.

Components Guide – beş değiştirilebilir yüzey

Model Clients	05:1984	sağlayıcıya konuşan şey
Model Context	05:2341	modele NE gideceğine karar veren şey
Tools	05:2473	modelin çağırabildiği fonksiyonlar
Workbench (+ MCP)	05:2841	tool'ların toplandığı arayüz
Code Executors	05:3054	Kodu nerede koşturacağın

Component config
05:1888

dump_component / load_component
her bileşen JSON'a yazılıp
geri yüklenebiliyor
→ yapılandırma kod değil VERİ

Beşi de değiştirilebilir yüzey. Bir ajan bunların hangi uygulamasıyla konuştuğunu bilmiyor — kapıyı kurmayı mümkün kılan da bu.

core kılavuzu 05:1977 – Components Guide

core'un "bileşen" dediği şey, ajan döngüsünün sökölüp değiştirilebilen parçaları. Beş tane var. **Model client** sağlayıcıyla konuşuyor. **Model context** modele ne gideceğine karar veriyor. **Tools** modelin çağırabildiği fonksiyonlar. **Workbench** o tool'ların toplandığı arayüz. **Code executors** kodun nerede koşacağını belirliyor. Beşinin ortak özelliği şu ve mimarının kilidi burada: **ajan, hangi uygulamayla konuştuğunu bilmiyor**. Onay kapısını bir workbench sarmalayıcısı olarak kurabilmemizin sebebi tam olarak bu. Ayrıca hepsi `dump_component` ile JSON'a yazılıp geri yüklenebiliyor: yapılandırma kod değil, **veri**.

Çok-ajan desenleri – resmî sekizi

Concurrent Agents	05:3236	tek yayın → çok dal → toplayıcı
Sequential Workflow	05:3504	her ajan bir sonrakine devrediyor
Group Chat	05:3772	bir yönetici konuşma sırasını dağıtıyor
Handoffs	05:4349	ajan işi kendisi devrediyor
Mixture of Agents	05:4989	aynı soru, farklı uzmanlar, birleştirici
Multi-Agent Debate	05:5358	birden çok tür karşılıklı eleştiri
Reflection	05:5822	üretici + eleştirmen, kalite döngüsü
Code Execution	05:6188	modelin yazdığı kod yürütücüde koşuyor

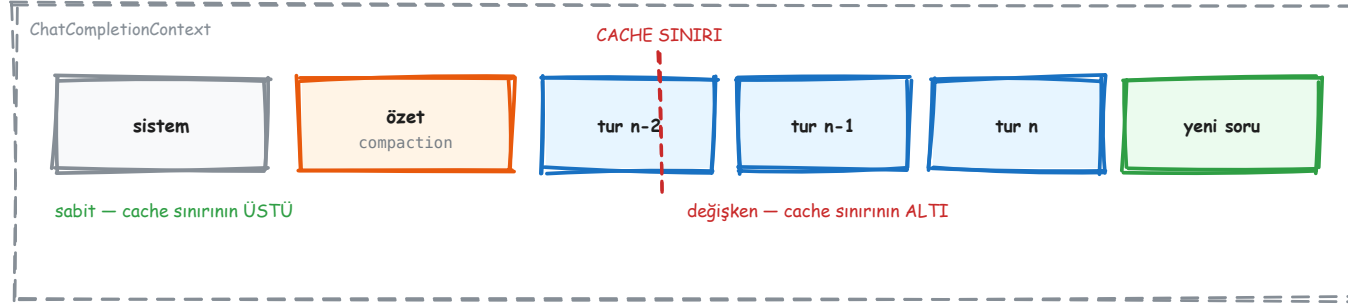
Bu, core kılavuzunun KENDİ bölümlmesi — bizim tasnifimiz değil. Satır numarası verilmesinin sebebi bu.

core kılavuzu 05:3206 – Multi-Agent Design Patterns

Kılavuzun kendi bölümlmesinde sekiz başlık var. Yedisi orkestrasyon deseni; sonuncusu (**Code Execution**) bir orkestrasyon değil, bir yetenek. Asıl dikkat edilecek şey listede **olmayanlar**.

Planlayıcı-yürütücü, kara tahta, yönlendirici gibi desenler literatürde var ama core kılavuzunda yok. Yani bir kaynakta "AutoGen'in dokuz deseni" diye bir tablo görürsen, o tasnif kılavuzun değil **yazarın**. Bunu kontrol etmenin yolu da açık: her başlığın 05 satır numarası var. Bizim fiilen kullandığımız iki tane: **Concurrent Agents** (tarama boru hattı) ve kısmen **Reflection** (risk denetçisi üç analizi çapraz kontrol ediyor).

Bağlam ve cache sınırı



sabit olan önde → önek önbelleğe uygun kalır

Model hiçbir şey hatırlamıyor. Her turda bağlamın tamamı baştan gönderiliyor. "Hatırlıyor" dediğimiz şey işte bu tekrar gönderme, ve her tur bir öncekinden pahalı. Sağlayıcılar bunu ucuzlatmak için istegin başındaki *değişmeyen* kısmı çok daha düşük ücretlendiriyor. Ama önbellek **önekten** çalışıyor: baştan başlayıp ilk farklı bayta kadar. Dolayısıyla değişken bir şeyi başa koyarsan, arkasındaki her şey önbellekten düşüyor — tek bir tarih damgası bütün sistem prompt'unu yakabilir. Sorulacak doğru soru "prompt nasıl kısalır" değil, **"ne gerçekten sabit kalabilir"**.

Durmayı öğretmek



sert tavan modelden bağımsız · anlamsal koşul modele bağımlı

On bir sonlandırma koşulu var, ama pratikte ikiye ayrılıyorlar. **Sert tavanlar** — mesaj sayısı, token bütçesi, geçen süre — modelden tamamen bağımsız çalışıyor ve her zaman tutuyor. **Anlamsal koşullar** — "model BİTTİ yazınca dur" gibi — modelin işbirliğine bağlı. Model o kelimeyi yazmazsa koşul hiç tetiklenmiyor. Buradan çıkan kural net: üretimde her zaman **en az bir sert tavan** olmalı. Anlamsal koşul iyi bir kolaylık ama tek başına bir güvence değil. Tavansız koşan bir takım, faturayı modelin kararına bırakır.

Dört sessiz varsayılan

<code>max_tool_iterations</code> varsayılan: 1	model tool sonucunu GÖRMEDEN cevaplar
<code>model_context</code> varsayılan: yok	ajanın belleği yoktur
<code>model_client_stream</code> varsayılan: False	token akışı hiç yayılmaz
<code>sonlandırma</code> varsayılan: yok	takım tavansız koşar

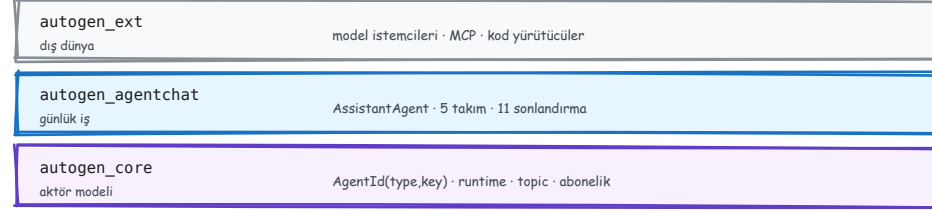
Dördünün ortak yanı: **HIÇBİRİ HATA VERMEZ.**

Sistem çalışır, sonuç yanlış olur — ve aramak için önce yanlış olduğunu bilmen gerekir.

hiçbiri hata vermiyor

Bir çerçevede en pahalı şey, makul görünen ama sessizce yanlış olan varsayılandır. AutoGen'de dört tane var, ve dördü de sistemi çalıştırıp sonucu bozuyor. `max_tool_iterations=1` tool sonucunu modelden saklıyor. `model_context` vermezsen ajanın belleği hiç olmuyor. `model_client_stream=False` ile token akışı hiç yayılmıyor. Sonlandırma koşulu yazmazsan takım tavansız koşuyor. Dördünü de **açıkça yazmak** iki şey kazandırıyor: bugün ne olduğunu bilirsin, ve bir sonraki sürümde varsayılan değişirse kodun etkilenmez.

Geriye kalan



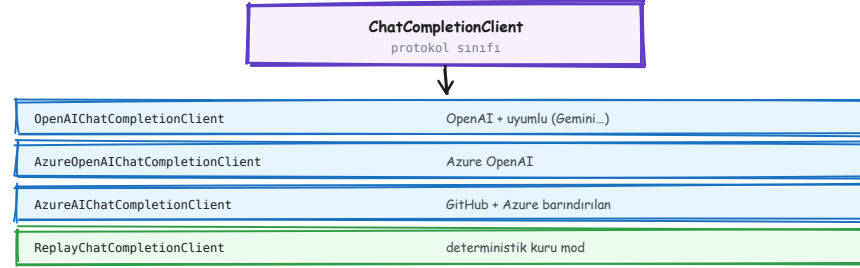
Ayıran şey en alt katman: ajanlar gerçekten aktör — kendi mailbox'ı olan, mesajı tipe göre yönlendiren birimler.

Yukarıdan başla; aşağı inmek her zaman mümkün.

motor hazır, kuşatma değil

AutoGen sana bir **motor** veriyor: aktör modeli, takımlar, akış, sonlandırma, olay yayını. Bunlar sağlam, ve bu destede gördüğün her şey gerçekten çalışıyor. Vermediği şey **kontrol düzlemi**, ve üçünün de yakını var ama üçü de eksik. **Kapı** mesaj katmanında duruyor: bir mesajı düşürebiliyor ama "bu ajan şu komutu çalıştırmak istiyor"u görmüyor, ve reddi ajana gerekçesiz dönüyor. **Onay** var — yalnız `CodeExecutorAgent` içinde, deneysel, kod çalıştırmaya özel, verilmezse sadece uyarı; ve onaylı bir ajan **yapılandırmaya yazılamıyor**, kod bunu reddediyor. **Denetim kaydı** ise Python logging'i: teslim garantisi yok, ve modelin bütün mesajlarını içine koyuyor. İkinci deste aynı problemi *ters uçtan* çözmüş bir sistem — ve onun denetim kaydının sorunu tam tersi olacak: hiç içerik tutmuyor.

Model Clients

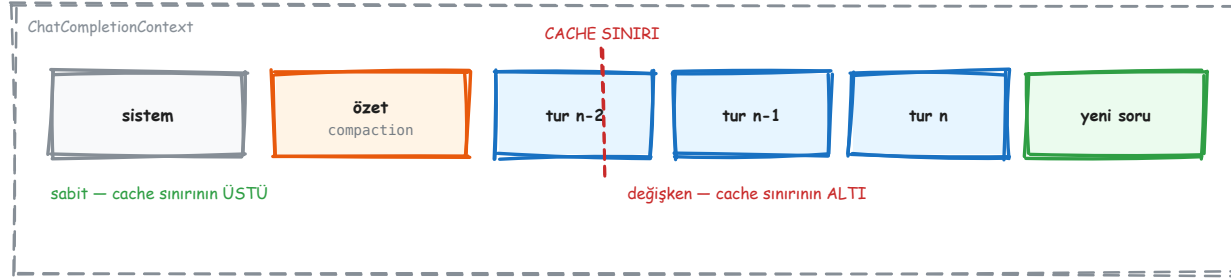


Hepsi aynı protokolü uyguluyor — ajan hangisiyle konuştuğunu bilmiyor. Kuru mod istemcisi de bir istemci.

05:1984 — dördü de aynı ChatCompletionClient protokolünü uyguluyor

Model istemcisi, sağlayıcıyla konuşan katman. Ajan "şu mesajları gönder, cevabı getir" diyor; nasıl gönderileceğini istemci biliyor. Dört yerleşik istemci var: **OpenAI ve uyumlular** (Gemini dahil), **Azure OpenAI, Azure ve GitHub'da barındırılan** modeller, bir de **ReplayChatCompletionClient**. Sonuncusu gerçek bir modele hiç bağlanmıyor; önceden yazdığın cevapları sırayla döndürüyor. Ama diğerleriyle aynı protokolü uyguladığı için ajan aradaki farkı göremiyor. Bunun pratik değeri büyük: **test ve kuru mod için ayrı bir kod yolu yazmıyorsun**. Bu depodaki `engine.Ledger` tam olarak bunu kullanıyor, ve API anahtarı yokken tarama uçtan uca deterministik koşuyor.

Model Context



05:2341 — modele NE gideceğine karar veren şey

Model istemcisi sağlayıcıyla *nasıl* konuşulacağını bilir. Model context ise *ne* söyleneceğine karar verir. Mesajları saklıyor ve her model çağrısından önce sıralı listeyi veriyor. Yerleşik dört uygulamanın dördü de aynı şeyi yapıyor: fazlasını **kırpmıyor**. `Unbounded` hiç kırpmıyor, `Buffered` son n mesajı tutuyor, `HeadAndTail` baş ve son kısmı tutup ortayı atıyor, `TokenLimited` token limitine kadar tutuyor. Yani AutoGen'de **özetleyerek sıkıştıran hazır bir uygulama yok**. Sıkıştırma istiyorsan yazacaksın; biz de öyle yaptık. Ve buradaki tuzak sessiz: `model_context` vermezsen ajanın belleği hiç olmuyor, hiçbir hata da çıkmıyor.

Tools

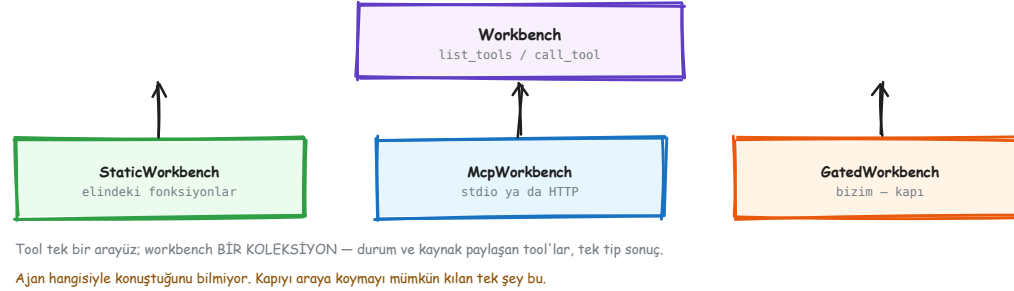


Tool bir kod parçası; ajan onu modelin ürettiği fonksiyon çağırısına karşılık koşturuyor. Yazdığın açıklama, arayüzün kendisi.

05:2473 – şema, fonksiyon imzasından ve docstring'den türetiliyor

Tool, ajanın bir eylem yapmak için koşturduğu koddur. Bir hesap makinesi kadar basit ya da bir üçüncü taraf API çağrısı kadar karmaşık olabilir. Modelin kendisi kodu çalıştırmıyor. Model yalnızca "şu tool'u şu argümanlarla çağır" diyen bir çıktı üretiyor; çalıştırmayı çerçeve yapıyor. Şaşırtıcı olan şu: tool'un işe yarayıp yaramaması büyük ölçüde **koda değil, açıklamaya** bağlı. Model, bu tool'u ne zaman çağıracağına docstring'i okuyarak karar veriyor; ne göndereceğine de tip ipuçlarına bakarak. Tip ipucu yazmazsan model tahmin eder. Yani tool yazmanın yarısı Python, yarısı **arayüz tasarımı**.

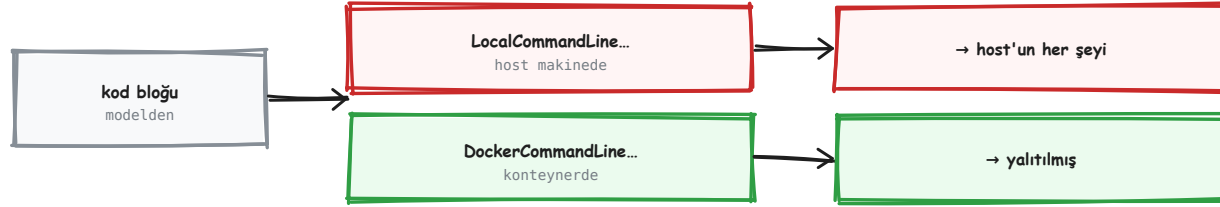
Workbench (ve MCP)



05:2841 — tek tool değil, durum ve kaynak paylaştan bir koleksiyon

Tool tek bir tool'a arayüz verir. **Workbench** ise birden çok tool'a birden verir, ve o tool'lar **durum ve kaynak paylaşır**. Fark neden önemli? Çünkü bir MCP sunucusuna açılan *tek* bağlantı, o sunucudaki bütün tool'ları taşıyabiliyor. Her tool için ayrı bağlantı gerekmiyor. Üç uygulama var: **StaticWorkbench** elindeki Python fonksiyonlarını sarıyor, **McpWorkbench** stdio ya da HTTP üstünden uzak bir sunucuya bağlanıyor, ve **GatedWorkbench** **bizim eklediğimiz** onay kapısı. Ajan açısından üçü de aynı: `list_tools()` ve `call_tool()`. **Kapıyı araya koyabilmemizin tek sebebi bu.**

Code Executors



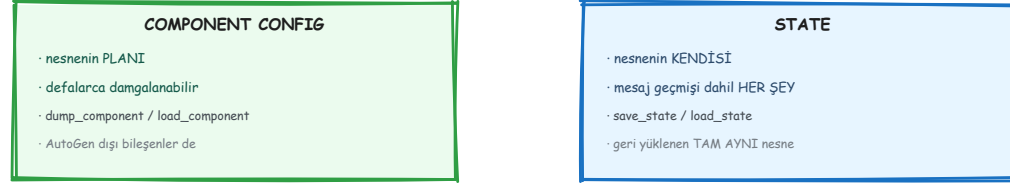
Her kod bloğu bir dosyaya yazılıp AYRI BİR SÜREÇTE koşuyor — yani bloklar arası değişken paylaşımı yok.

Local, modelin yazdığı kodu makinede koşturur. Bu bir tercih değil, bir güven kararıdır.

05:3054 — her blok bir dosyaya yazılıp AYRI SÜREÇTE koşuyor

Modelin yazdığı kodu çalıştırmanın en basit biçimi şu: her kod bloğu bir dosyaya kaydediliyor, sonra o dosya çalıştırılıyor. Bunun doğrudan bir sonucu var ve şaşırtıyor — **bloklar arasında değişken paylaşımı yok**. Her blok yeni bir süreç olduğu için ilk blokta tanımladığın değişken ikincisinde yok. İki uygulama var ve aralarındaki fark teknik değil, **güvenle ilgili**. `DockerCommandLineCodeExecutor` kodu bir konteynerde koşturuyor. `LocalCommandLineCodeExecutor` ise **senin makinende**. İkincisi, bir dil modelinin yazdığı kodu kendi diskine erişimle çalıştırmak demek. Bu bir performans tercihi değil, bir **güven kararı**, ve varsayılan olarak seçilmemesi gereken taraf.

Component config – plan ile fotoğrafın farkı

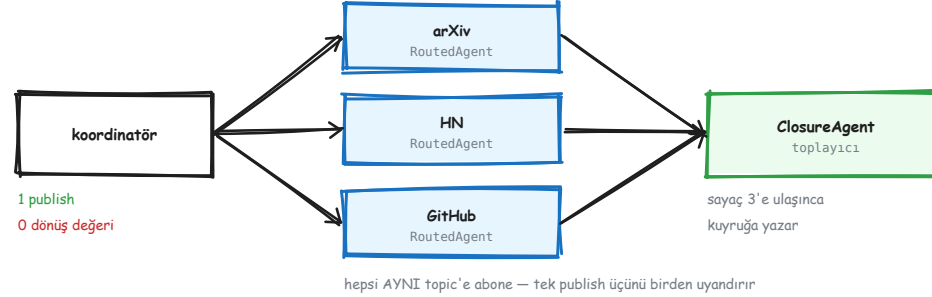


Kılavuzun kendi vurgusu: config bir <plan>, state bir <fotoğraf>. Aynı config'ten yüz örnek çıkar; aynı state'ten tek bir nesne.
 AutoGen Studio'nun yapılandırma tabanlı deneyimi bunun üstünde.

05:1888 – config bir PLAN, state bir FOTOĞRAF

Kılavuzun kendi ayrımı, ve karıştırılması çok kolay. **Component config** bir nesnenin *planı*: hangi sınıf, hangi ayarlarla. `dump_component` ile JSON'a yazılıyor, `load_component` ile geri kuruluyor. Aynı plandan istediğin kadar yeni örnek damgalayabiliyorsun. **State** ise nesnenin o anki hâli: mesaj geçmişi dahil her şey. Bunu geri yüklerken **tam olarak aynı** nesneye yüklemek zorundasın, çünkü state kendi başına ne olduğunu bilmiyor. Pratik sonucu şu: yapılandırma artık kod değil **veri**. AutoGen Studio'nun tıklayarak ajan kurma deneyimi bunun üstünde duruyor, ve AutoGen dışında tanımlanmış bileşenler de aynı sisteme katılabiliyor.

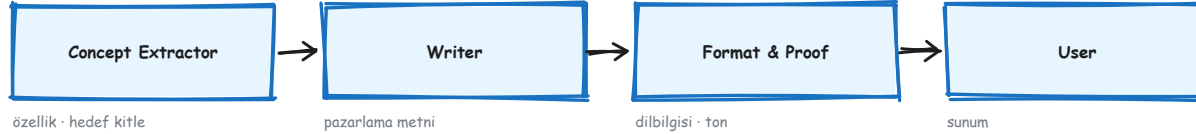
Concurrent Agents



05:3236 — eşzamanlılık abonelikten geliyor, bir çağrıdan değil

Kılavuz bu deseni üç alt başlıkta anlatıyor. **Tek mesaj, çok işleyici**: aynı topic'e abone birden çok ajan aynı mesajı aynı anda işliyor. **Çok mesaj, çok işleyici**: mesaj tipleri topic'lere göre ayrı ajanlara gidiyor. Bir de **doğrudan mesajlaşma**. Bizim tarama boru hattımız birincisini kullanıyor. Ve ölçtüğümüz kardeş kaybı problemi tam burada yaşıyor: dallardan biri sessizce ölürse toplayıcı sonsuza kadar bekliyor. Sebebi mekanik — `publish_message` hata fırlatmıyor, yalnız logluyor. Toplayıcı de eksik dalın öldüğünü öğrenemiyor. Çözüm küçük ama zorunlu: **sayacı finally bloğunda artır**. Bu desenin fiyatı o satır.

Sequential Workflow

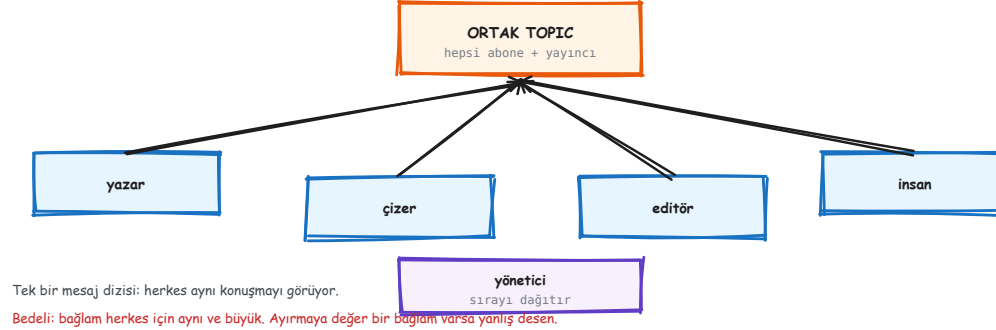


Sıra DETERMİNİSTİK: her ajan bir alt görevi yapıp bir sonrakine devrediyor. Kim konuşacak diye kimse karar vermiyor.
core'da bu, her ajanın bir sonrakinin topic'ine yayın yapmasıyla kuruluyor — zincir aboneliklerde yazılı.

05:3504 — deterministik sıra, her ajan bir alt görev

Ajanlar önceden belli bir sırayla çalışıyor. Her biri bir mesajı alıyor, işliyor, çıktısını bir sonrakine veriyor. Kimin konuşacağına **hiç kimse karar vermiyor**, çünkü sıra zaten kodda yazılı. Kılavuzun örneği dört ajanlı bir pazarlama hattı: **Concept Extractor** ürün açıklamasından özellikleri ve hedef kitleyi çıkarıyor, **Writer** pazarlama metnini yazıyor, **Format & Proof** dilbilgisini ve tonu düzeltiyor, **User** sonucu sunuyor. core katmanında bu, her ajanın bir sonrakinin topic'ine yayın yapmasıyla kuruluyor. Yani **zincir aboneliklerde yazılı**, ortada zinciri yöneten bir orkestratör yok. Sıra belliyse bu en ucuz desen: fazladan hiç model çağrısı yapmıyor.

Group Chat



05:3772 – herkes aynı topic'e hem abone hem yayıncı

Bir grup ajan **ortak bir mesaj dizisini** paylaşıyor. Hepsi aynı topic'e abone, hepsi aynı topic'e yayın yapıyor. Yani her ajan, diğerlerinin yazdığı her şeyi görüyor. Katılımcılar farklı işlerde uzman oluyor: yazar, çizer, editör gibi. İstersen ajanları yönlendirmesi için araya bir **insan katılımcı** da koyabiliyorsun. Gücü de bedeli de aynı yerden geliyor. Herkes aynı konuşmayı gördüğü için hiçbir bilgi kaybolmuyor. Ama **bağlam da herkes için aynı ve büyük**: her ajanın her turu, bütün konuşmanın maliyetini ödüyor. Ayrılmaya değer bir bağlam varsa bu yanlış desen. O durumda fan-out doğru olan.

Handoffs



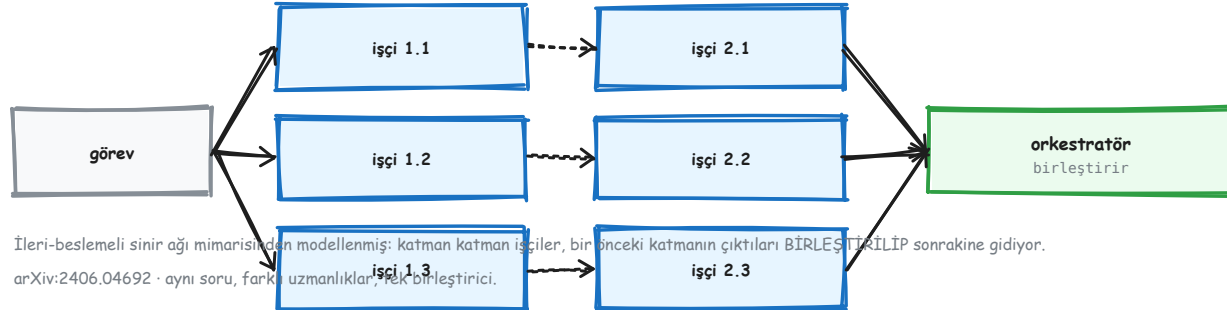
OpenAI'ın Swarm projesinden geliyor. AutoGen'in eklediği: dağıtık runtime'a ölçeklenebilmesi ve kendi ajan uygulamanı getirebilmen.

05:4349 — devretme, özel bir tool çağrısından ibaret

Fikir OpenAI'ın **Swarm** adlı deneysel projesinden geliyor. Bir ajan, görevi başka bir ajana devrediyor; ve devretme özel bir tool çağrısı olarak yapılıyor. Dışarıda kimi kimin izleyeceğine karar veren bir yönlendirici yok. Devretme kararını **ajanın kendisi** veriyor. AutoGen bunun üstüne üç şey eklemiş: dağıtık runtime'a ölçeklenebilmesi, kendi ajan uygulamanı getirebilmen, ve doğal async API.

Ölçtüğümüz sonuç: bu, karşılaştırdığımız dört desen içinde en pahalısı. 334 token, SelectorGroupChat'in **%63,7 üstünde**. Sebebi de mekanik: her devirde bağlam yeniden kuruluyor. Ödediğin şey zekâ değil, **yönlendirme özerkliği**.

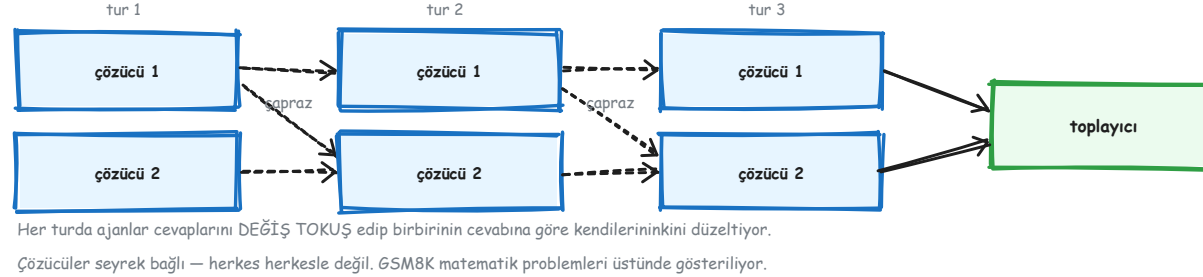
Mixture of Agents



05:4989 – ileri-beslemeli bir sinir ağından modellenmiş

İki tip ajan var: bir sürü **işçi** ve tek bir **orkestratör**. İşçiler katmanlara ayrılmış ve her katmanda sabit sayıda işçi var. Bir katmandaki işçilerin çıktıları **birleştirilip** sonraki katmandaki bütün işçilere gönderiliyor. Sinir ağı benzetmesi bizim değil, kılavuzun kendi ifadesi. Pratikte olan şu: aynı soruya farklı uzmanlıklar bakıyor, sonra bir birleştirici hepsini tek cevaba indiriyor. İşe yaradığı yer, uzmanlık alanlarının **gerçekten ayrı** olduğu durumlar. Ayrı değilse aynı cevabın pahalı kopyalarını üretmiş olursun. Kaynak makale: arXiv:2406.04692.

Multi-Agent Debate



05:5358 — her turda cevaplar karşılıklı değiş tokuş ediliyor

Çok turlu bir etkileşim. Her turda ajanlar cevaplarını birbirine gösteriyor, sonra *diğerlerinin cevaplarına bakarak* kendi cevabını düzeltiyor. İki tip ajan var: **çözücüler** ve tek bir **toplayıcı**. Ve bir tasarım ayrıntısı önemli: çözücüler **seyrek** bağlanmış. Herkes herkesle konuşmuyor. Bu kasıtlı, çünkü tam bağlı bir grup hem pahalı hem de hızla tek bir görüşe yakınsıyor. Kılavuz bunu GSM8K matematik problemleri üstünde gösteriyor, yani **cevabı doğrulanabilen** bir alanda. Bu ipucu önemli: münazara, yanlışın tespit edilebildiği yerlerde işe yarıyor. Görüş meselelerinde yalnızca daha uzun konuşuyorsun.

Reflection



İkinci LLM üretimi, birincinin ÇIKTISINA koşullanmış. Döngü eleştirmen tatmin olana kadar sürüyor.

Bizde karşılığı: RiskAuditor — üç analizi çelişki ve kaynaklı iddia için çapraz kontrol ediyor.

05:5822 — ikinci üretim, birincinin çıktısına koşullanmış

Bir model çıktı üretiyor, ardından ikinci bir model o çıktının kritiğini yazıyor. İkinci üretim birincinin sonucuna koşullanmış olduğu için ona "yansıtma" diyor. Kod yazma örneğinde: birinci model kodu yazıyor, ikincisi kodu inceleyip neyin yanlış olduğunu söylüyor. Ajan dünyasında bu bir **çift** olarak kuruluyor. Biri mesaj üretiyor, diğeri o mesaja cevap veriyor, ve kritik tatmin olana kadar konuşma devam ediyor. Bu depodaki karşılığı **RiskAuditor**. Klasik yansıtmadan bir farkı var: kendi çıktısını değil, **üç analistin çıktısını** denetliyor. Aradığı şey çelişki ve kaynak gösterilmemiş iddia.

Code Execution

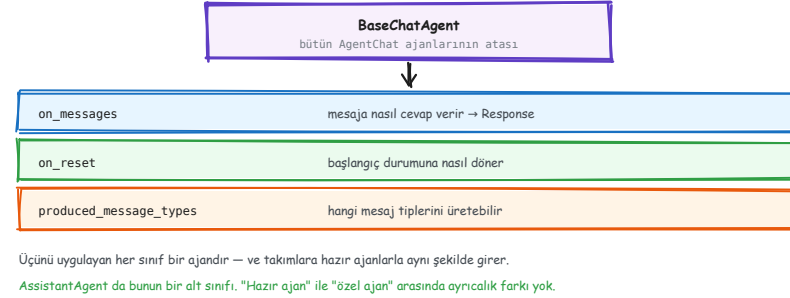


İki ayrı ajan, tek bir Message veri sınıfı. AgentChat'te hazır karşılığı var (CodeExecutorAgent) ama kılavuz burada elle yazmayı gösteriyor.
Kılavuzun örneği: Tesla ve Nvidia hisse getirilerinin grafiğini çizdirmek.

05:6188 — kodu yazan ajan ile koşturan ajan ayrı

Sekizincisi aslında bir orkestrasyon deseni değil, bir **yetenek**. İki ajan var: **Assistant** kodu yazıyor, **Executor** koşturuyor. Aralarında tek bir **Message** veri sınıfı gidip geliyor. AgentChat'te bunun hazır karşılıkları da var (**AssistantAgent** , **CodeExecutorAgent**). Ama kılavuz bilerek elle yazmayı gösteriyor, çünkü amaç hazır sınıfı tanıtmak değil; **hafif bir özel ajanın nasıl yazıldığını** göstermek. Örneği somut: Tesla ile Nvidia hisse getirilerinin grafiğini çizdirmek. Ve hangi yürütücüyü seçtiğin bir güven kararı — bir önceki bileşen slaytındaki Docker/local ayrımı.

Custom Agents



08:7248 — BaseChatAgent + üç üye, hepsi bu

Hazır ajanlar istediğin davranışı vermiyorsa kendi ajanını yazıyorsun. Sözleşme şaşırtıcı derecede küçük: BaseChatAgent 'tan türeyip üç şey uyguluyorsun. **on_messages** gelen mesaja nasıl cevap vereceğini söylüyor ve bir Response döndürüyor. **on_reset** ajanı başlangıç durumuna döndürüyor. **produced_message_types** hangi mesaj tiplerini üretebildiğini bildiriyor. Bunu bilmenin asıl değeri şu: AssistantAgent da bu sınıfın bir alt sınıfı. Yani "hazır ajan" ile "özel ajan" arasında **ayrıcalık farkı yok**. Senin yazdığın ajan da takımlara giriyor, akışa katılıyor, sonlandırma koşullarına takılıyor — hepsi aynı şekilde.

Memory ve RAG



Kritik olan üçüncü metod: `update_context`, ajanın KENDİ `model_context`'ini değiştiriyor — yani getirilen bilgi bağlama model çağrısından hemen önce giriyor.

Uygulamalar: `ListMemory` (basit liste) · `ChromaDBVectorMemory` · `Redis`. Klasik RAG deseni bu protokolün bir örneği.

08:6242 — `add` · `query` · `update_context` · `clear` · `close`

Bir olgu deposu tutup, onu **belirli bir adımdan hemen önce** ajanın bağlamına eklemek istediğin durumlar için. Klasik RAG deseni bunun bir örneği: sorgu bir veritabanından ilgili bilgiyi getiriyor, o bilgi bağlama giriyor. Protokolün beş metodu var ama **kritik olan üçüncüsü**. `update_context`, ajanın *kendi* `model_context` 'ini değiştiriyor. Bunun anlamı önemli: getirme bir tool çağrısı **değil**. Model "bilgi getir" diye bir şey çağırıyor; bağlam, model çağrısından hemen önce sessizce zenginleşiyor. Model neyin nereden geldiğini görmüyor. Hazır uygulamalar: `ListMemory`, `ChromaDBVectorMemory`, `Redis`.

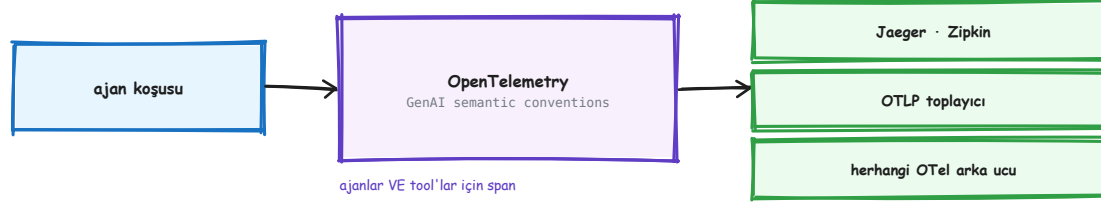
Serializing Components – ve tek sert uyarı



08:6834 — `dump_component()` / `load_component()`

Ajan, takım, model istemcisi — hepsi bildirimsel bir şartnameye yazılıp geri kurulabiliyor. Hata ayıklamak, görselleştirmek ve **çalışmanı başkasıyla paylaşmak** için. Ama AgentChat kılavuzunun bütün bölümü içindeki **tek büyük harfli uyarı** burada duruyor, ve sebebi mekanik: her bileşen kendi serileştirme mantığını kendisi yazıyor. Bir nesneyi geri kurmak **kod çalıştırmayı içerebiliyor**; serileştirilmiş bir fonksiyon buna örnek. Sonuç açık: paylaşılan bir takım JSON'u, paylaşılan bir betikle **aynı güven sınıfında**. Bir kurumda bunu "yapılandırma dosyası" diye elden ele dolaştırmak, kod dağıtmakla aynı şey.

Tracing ve Observability



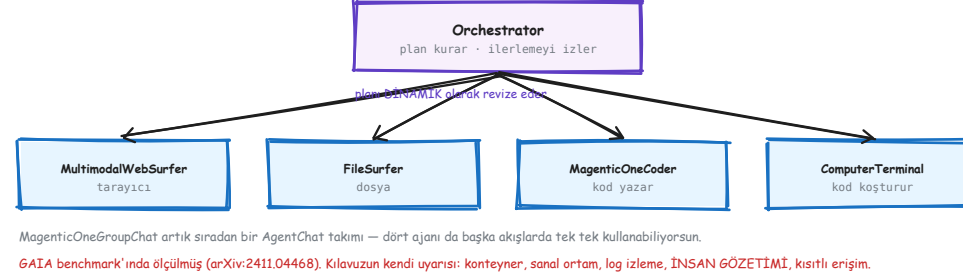
AutoGen'in tracing'i yerleşik ve OpenTelemetry'ye dayanıyor — yani arka ucu sen seçiyorsun, çerçeve dayatmıyor.

GenAI Semantic Conventions'ı takip ediyor (hâlâ geliştirilme aşamasında). Bizde KURULU DEĞİL — olay akışıyla yetiniyoruz.

08:6975 — OpenTelemetry, GenAI Semantic Conventions

Logging sana tek tek olayları verir. **Tracing** ise bir koşunun *yapısını* verir: hangi ajan hangi tool'u ne kadar sürede çağırdı, hangi adım hangisinin içinde geçti. Yerleşik ve **OpenTelemetry**'ye dayanıyor. Yani arka ucu sen seçiyorsun: Jaeger, Zipkin, ya da herhangi bir OTLP toplayıcı. Çerçeve sana bir gözlem ürünü dayatmıyor. Ayrıca **GenAI Semantic Conventions**'ı takip ediyor. Bu hâlâ geliştirilmekte olan bir standart, ve anlamı şu: AutoGen'in ürettiği kayıtlar başka çerçevelerinkile aynı sözlüğü kullanıyor, dolayısıyla karşılaştırılabilir. **Bizde kurulu değil.** Olay akışıyla yetiniyoruz, ve bu bir eksik.

Magentic-One



08:6053 — orkestratör + dört uzman ajan, arXiv:2411.04468

Açık uçlu web ve dosya görevleri için kurulmuş **genelci** bir çok-ajan sistemi. Belirli bir işe göre değil, "ne gelirse" diye tasarlanmış. Orkestratör bir plan kuruyor, görevleri dağıtıyor, ilerlemeyi izliyor, ve tıkanıldığında planı **koşarken revize ediyor**. Dört uzman var: **MultimodalWebSurfer** tarayıcı kullanıyor, **FileSurfer** dosya okuyor, **MagenticOneCoderAgent** kod yazıyor, terminal ajanı da koşturuyor. Önce **autogen-core** üstüne yazılmış, sonra AgentChat'e taşınmış. Yani bugün **sıradan bir AgentChat takımı**, ve dört ajanı tek tek başka akışlarda da kullanabiliyorsun. Kılavuzun kendi uyarısı altı maddeli ve ciddi: konteyner kullan, sanal ortam kullan, logları izle, **insan gözetimi koy**, erişimi kısıtla, veriyi koru.